Data Structure Time Comparisons on School Data

Charlie Polonus

April 8, 2025

Introduction

Given a set of two different lists, one containing every school in the US, and the other containing every school just in Illinois, the goal was to test various data structures (linked list, binary search tree, hashmap), seeing how each of them compared when inserting school data, finding a school by its name, and deleting the school based on its name. Each data structure used a base School struct, which contained the school's name, address, city, state, and county.

Tests

There are three different data structures being tested:

- A Singly-Linked List
- A Binary Search Tree
- A Hashmap (Closed Addressing, Length of 54799)

There are also two different datasets being scanned:

- USA Schools (164463 items)
- Illinois Schools (7423 items)

The tests that each of the data structures will go through are as follows:

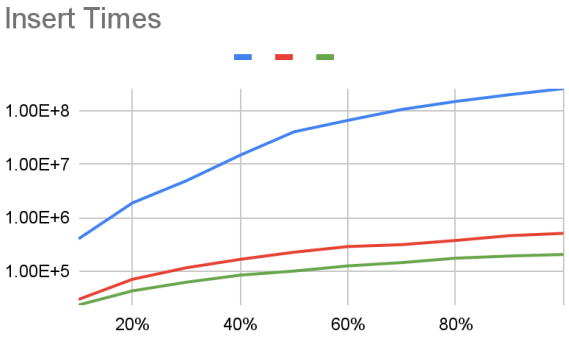
- Insertion
    - The data structure will need to insert all 164463 items from the USA school list into itself
- Searching
    - The data structure will need to find all 7423 Illinois schools from its container of all USA schools
    - It should be able to handle searching for schools that aren't in the container
- Deletion
    - The data structure will need to find and delete all 7423 Illinois schools from its container of USA schools
    - It should be able to handle searching for schools that aren't in the container

Each of those three tests will be run on each of the three data structures 10 different times, testing on different percentages of the total data, (10-100%) in order to test effectiveness at different lengths. All tests will be done in C++ using Clion.

Results (Linked List, Binary Search Tree (BST), Hashmap) Link to Full Spreadsheet

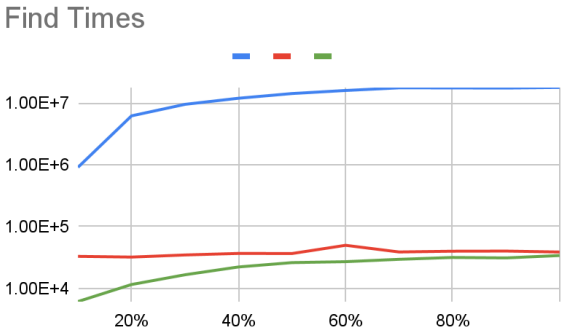
Insertion

Both the BST and hashmap stayed at roughly the same runtime, with hashmaps staying slightly below, while the linked list blew up after 20-30%. Despite both linked lists and closed addressing hashmaps having an insertion complexity of O(n), they have vastly different Thetas. (Θ(n) for linked lists, Θ(1) for hashmaps)



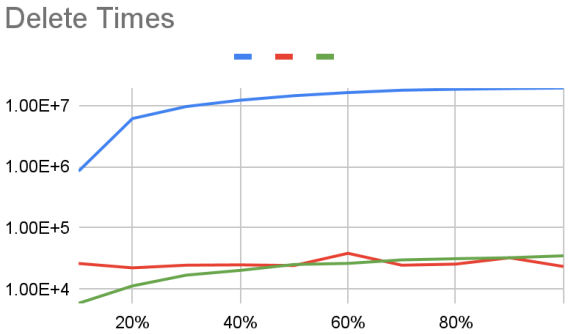
Searching

Similar to insertion, both the BST and hashmap seemed to stay around the same runtime, getting even closer as the number of items increased, with linked lists much higher. Interestingly, the complexity of the BST is around Θ(log(n)), while the hashmap is closer to Θ(1), despite them being so close to each other.



Deletion

Once again the trend continues of the BST and hashmap sharing a similar runtime, this time with both taking roughly the same time even just past 40%, with slight fluctuations. At complexities of Θ(log(n)) and Θ(1) respectively, they continue to be very closely timed to one another.



Challenges

Due to the large number of USA schools, paired with the slow insertion time of the linked list, (see Potential Improvements) iterating on the tests and the interface was slow, only being able to get through all tests in around roughly 10 minutes. Paired with oversights such as an extremely low set length of 23 for the hashmaps and a bug-ridden linked list implementation, progress started off slow, and only improved further into the project.

Potential Improvements

A large issue with the way that the linked list was implemented was that it only relied on a head node, and didn't take into account any tail nodes. This would've drastically reduced the insertion complexity to O(1), making it easily the fastest of the three. In addition, having the hashmap set

at a fixed length of 54799 regardless of data size likely limited it in certain cases, whereas having it use open-addressing/change fixed length based on data size might have been useful.

In terms of actual testing, there were a couple things that could've been done in terms of getting more accurate data. Using multiple different datasets with different kinds of information could've helped siphon out edge cases that might be present by just using the USA and Illinois school datasets. In addition, running each test multiple times and taking an average could've helped lessen the impact random fluctuations had on the tests. Lastly, preventing any other apps/programs from running and/or keeping the computational load of the computer stagnant for each test run would help weed out any potential interference in the amount of CPU being used by the rest of the computer.

Conclusion

After looking over the graphs and analyzing the different complexities, it seems very clear that the best data structure to use for this kind of problem would be either a hashmap or a binary search tree, at least when compared to a linked list. From that basic conclusion, the specific choice might then simply depend on how the user is planning on using the data. In the case of constant insertion/deletion, a hashmap might be more useful for its $\Theta(1)$ insertion/deletion time, whereas a binary tree might be more useful if the data is being accessed less often, for its more simplistic design and fewer variables.